

JUILLET 2025

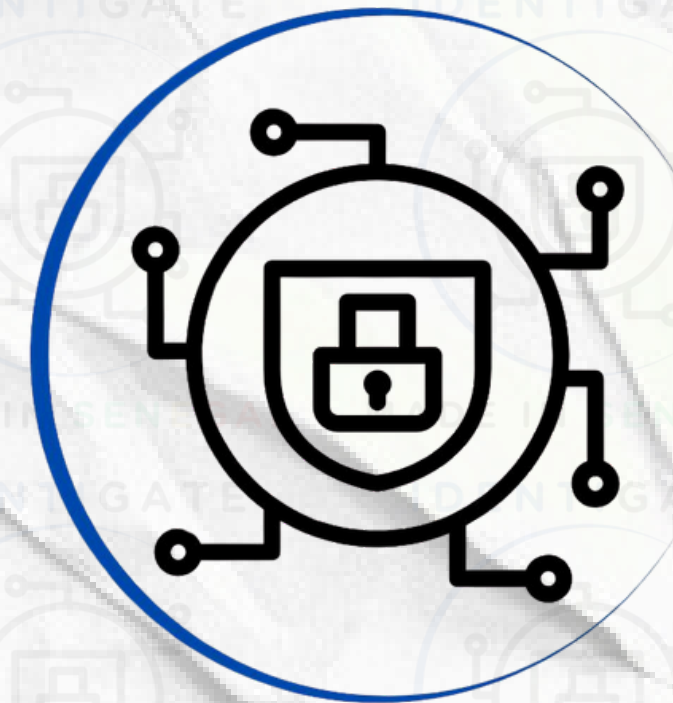
Gestion des Identités et Accès

Etude Comparative des
solutions IAM et Mise
en Œuvre

*Abdoul Ardo KAH - Ecole Centrale Polytechnique
d'Ingenieurs*

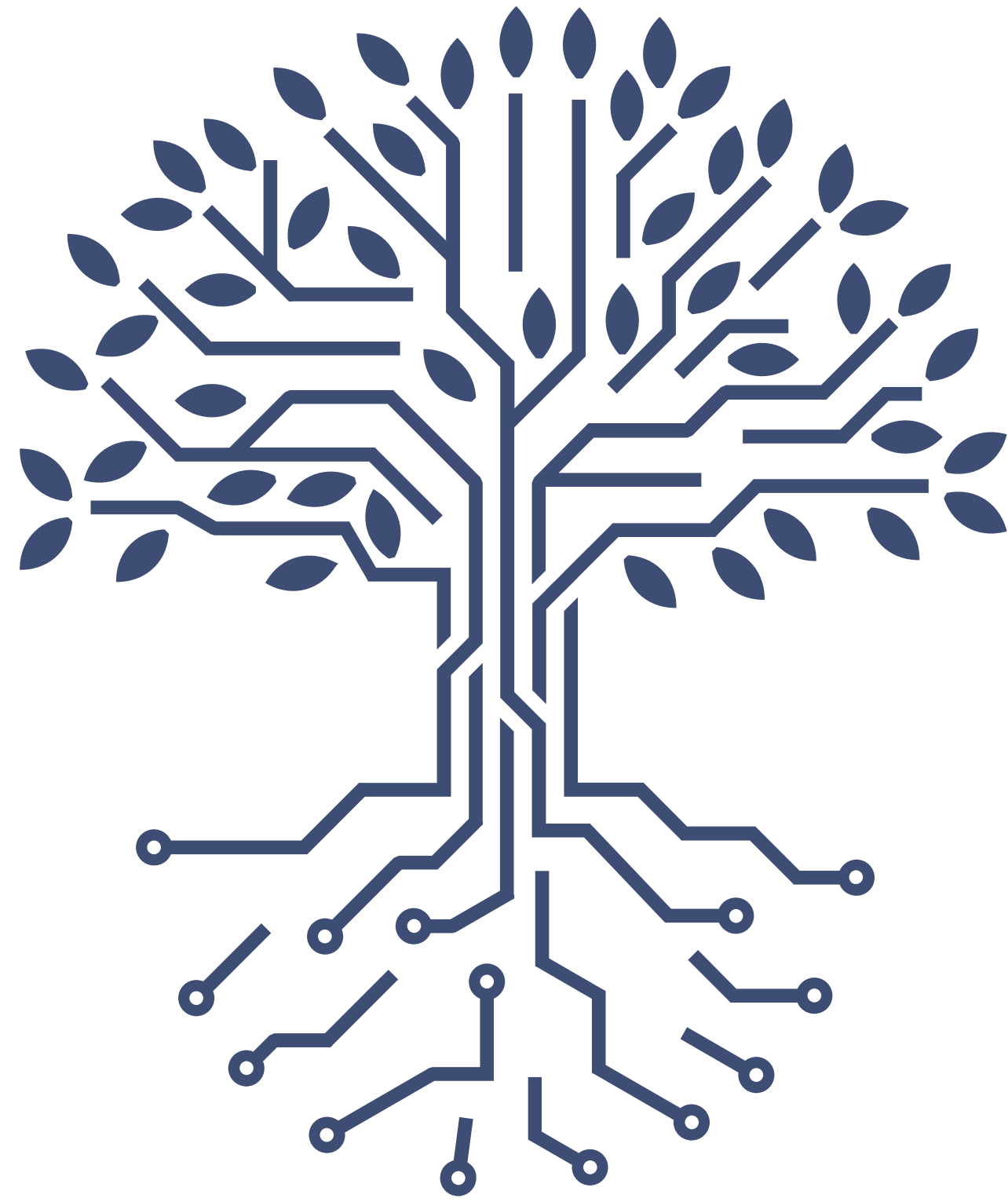


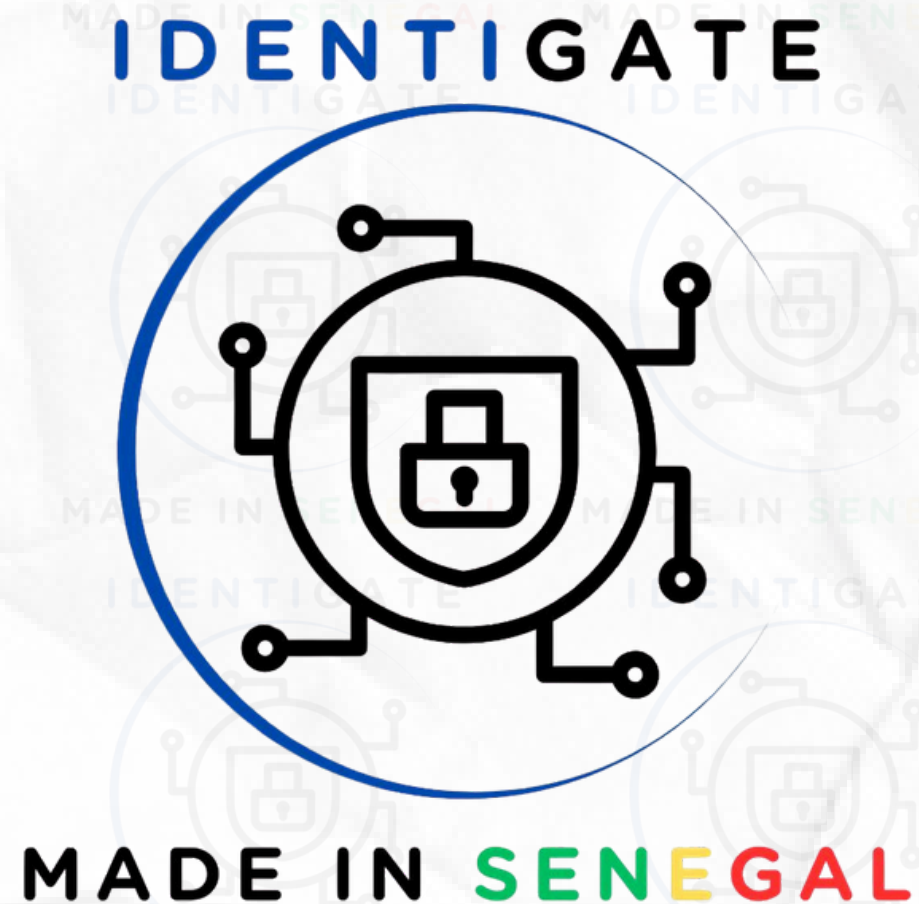
IDENTIGATE



MADE IN **SENEGAL**







Page 2 - 8

Introduction

Introduction et quelques definitions

I

Page 9 - 13

Problématique et état de l'art

Problématique - IAM -

Quelques solutions IAM existantes - Pourquoi le choix de Keycloak?

II

Page 13 - 18

Déploiement

Docker - Initialisation de l'environnement Keycloak

III

Page 26 - 31

Provisioning & Sécurité

Provisionnement automatique des utilisateurs - Mise en place de l'authentification forte(2FA) -

IV

Page 31-35

Gestion des rôles

Création et gestion de rôles personnalisés -
Assignation automatique via script - Affichage dynamique côté frontend

V

Page 36 - 38

Single Sign-On (SSO)

Intégration du SSO - Connexion centralisée
SSO - Déconnexion synchronisée
SSO

VI

Page 2-8

Export & Test

Export de la configuration- Tests avec différents profils utilisateurs

VII

Page 38-41

Conclusion

VIII



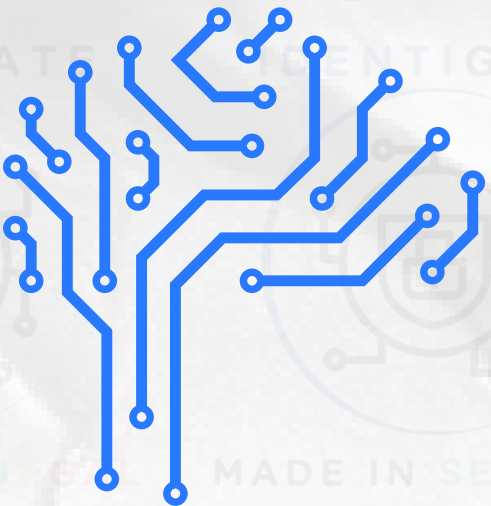
I

INTRODUCTION

Introduction et quelques definitions

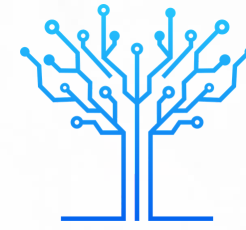


Gestion des Identités et
Accès



Introduction(1/2)

Introduction

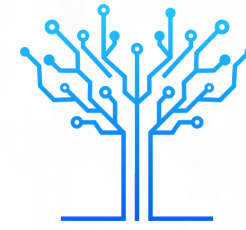


Dans un monde où la digitalisation est omniprésente, les entreprises doivent gérer de plus en plus d'utilisateurs, d'applications et de permissions. Garantir que seules les bonnes personnes accèdent aux bonnes ressources, au bon moment, est devenu un enjeu central de la cybersécurité.

Le projet IdentiGate répond à ce besoin en proposant une solution de gestion des identités et des accès (IAM) fondée sur des technologies open source, et en particulier sur Keycloak.

Introduction (2/2)

Introduction



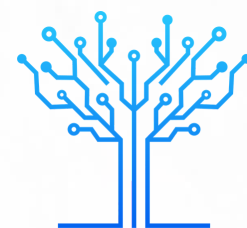
L'objectif est de mettre en place un système sécurisé permettant :

- l'authentification centralisée,
- le contrôle des rôles,
- le provisionnement automatisé des utilisateurs,
- ainsi que l'authentification forte via OTP.

À travers cette démarche, le projet vise à renforcer la sécurité d'un système d'information tout en simplifiant l'expérience des utilisateurs et des administrateurs.

Quelques définitions (1/2)

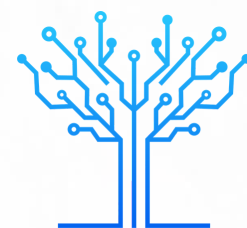
Introduction



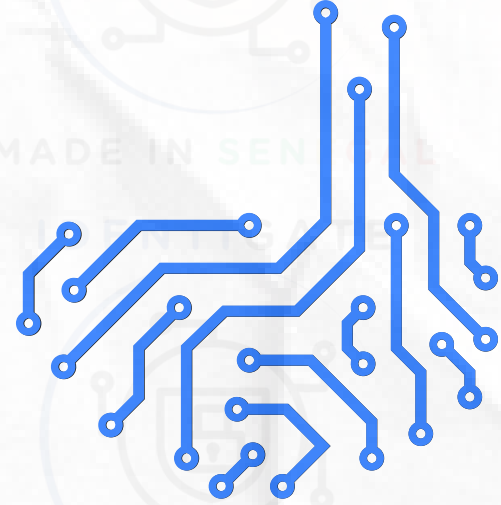
Terme	Definition
IAM (Identity Access Management)	Gestion centralisée des identités et des accès dans un système
Keycloak	Solution Open source de gestion des identités et d'authentification
SSO (Single Sign-On)	Connexion Unique pour accéder à plusieurs services/applications
Provisioning	Création automatique de comptes utilisateurs via un script ou
2FA/OTP	Authentification à deux facteurs avec code temporaire
Realm	Espace d'authentification isolé (Utilisateurs, rôles,clients)

Quelques définitions (2/2)

Introduction



Terme	Definition
OpenID	couche d'identité construite sur le protocole OAuth 2.0. Il permet à des applications de vérifier l'identité d'un utilisateur à l'aide d'un serveur d'autorisation, tout en obtenant des informations de profil de manière sécurisée (via des tokens).
SAML	protocole basé sur XML utilisé pour l'échange sécurisé d'informations d'identité entre un fournisseur d'identité (IdP) et un fournisseur de services (SP). Il est très utilisé dans les entreprises pour le SSO inter-applications
OAuth	protocole d'autorisation qui permet à une application d'accéder à des ressources au nom d'un utilisateur sans avoir à stocker ou manipuler son mot de passe. Il sert de base à OpenID Connect.



II

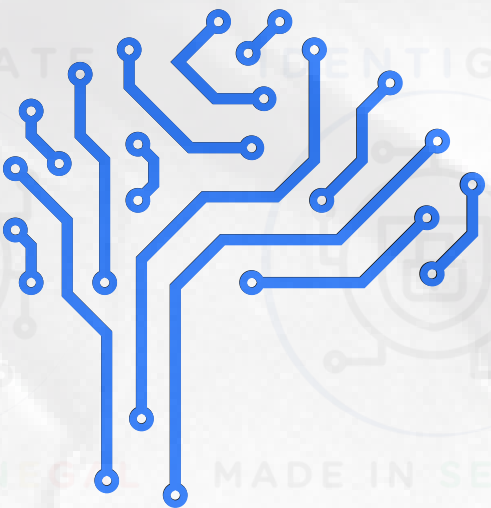
État de l'art

Problématique - IAM -

Quelques solutions IAM existantes - Pourquoi le choix de Keycloak?

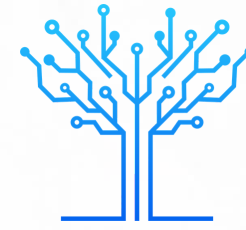


Gestion des Identités et
Accès



Problématique

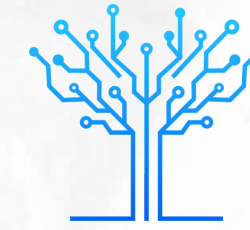
État de l'art



Dans un contexte numérique en constante évolution, la sécurité des accès est devenue un enjeu stratégique majeur pour les entreprises. Avec la multiplication des utilisateurs, des applications, des appareils et des accès distants, les risques de compromission d'identités ou d'usurpation sont plus élevés que jamais. Il est donc essentiel de mettre en place une gestion centralisée, fiable et sécurisée des identités et des accès.

L'IAM : Identity and Access Management

État de l'art

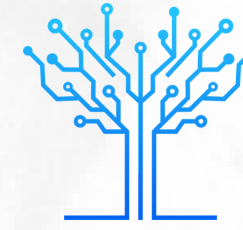


L'IAM (Identity and Access Management), ou gestion des identités et des accès, regroupe l'ensemble des processus, technologies et politiques permettant :

- de gérer qui peut accéder à quoi,
- de s'assurer que chaque utilisateur dispose des droits appropriés,
- et de garantir une traçabilité complète des accès.

Quelques solutions IAM existantes (1/2)

État de l'art



Il existe plusieurs solutions pour implémenter un système IAM, réparties en deux grandes familles :

- Solutions commerciales (propriétaires) comme :
 - Azure Active Directory (Microsoft)
 - Okta
 - IBM Security Identity Manager
- Solutions open source comme :
 - Keycloak
 - Gluu
 - OpenIAM

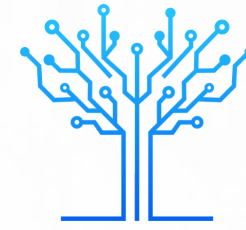
État de l'art



Tableau comparatif de quelques solutions Open Source & Commerciale

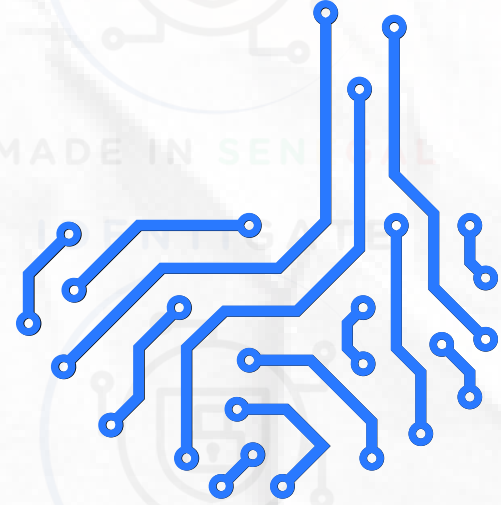
Pourquoi le choix de Keycloak?

État de l'art



Parmi les solutions open source, Keycloak s'impose comme une option particulièrement intéressante grâce à ses nombreux avantages :

- Gratuit et open source
- Interface complète de gestion
- Support natif du SSO (Single Sign-On) et du 2FA (authentification à deux facteurs)
- API REST puissante pour l'intégration avec d'autres systèmes
- Compatible avec Docker et Kubernetes pour une intégration cloud-native
- Prise en charge des protocoles standards : OpenID Connect, OAuth 2.0, SAML 2.0



III

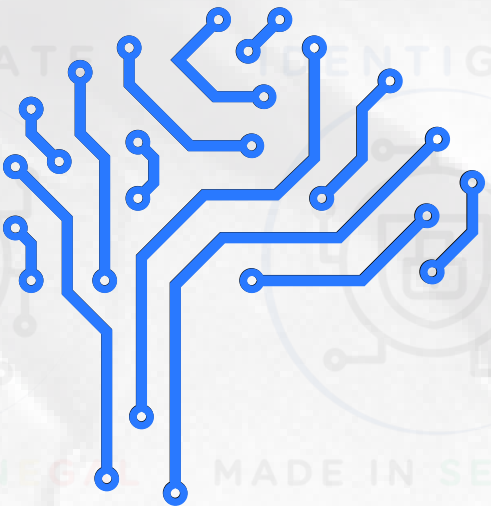
Déploiement

Architecture Technique

Docker - Initialisation de
l'environnement Keycloak



Gestion des Identités et
Accès



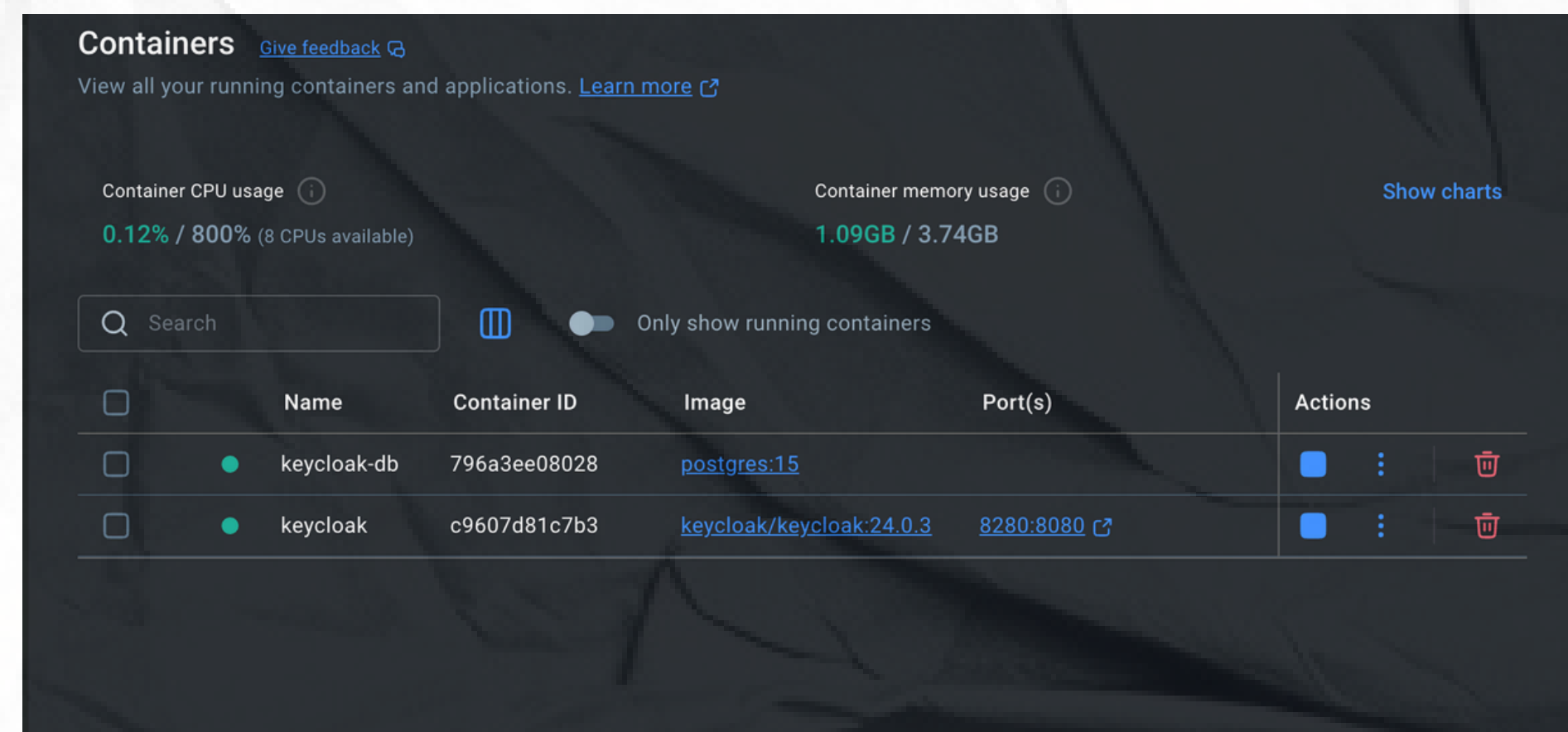
Utilisation de Docker pour un déploiement simplifié

Déploiement



Afin d'assurer un déploiement rapide, isolé et reproductible, Docker a été utilisé pour la mise en place de l'environnement IAM. L'infrastructure repose sur deux conteneurs principaux :

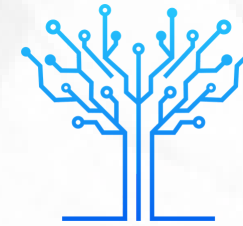
- Keycloak : le serveur d'authentification open source
- PostgreSQL : la base de données pour stocker les informations de configuration et d'identités



Containers Keycloak et Postgresql

Initialisation de l'environnement Keycloak(1/3)

Déploiement



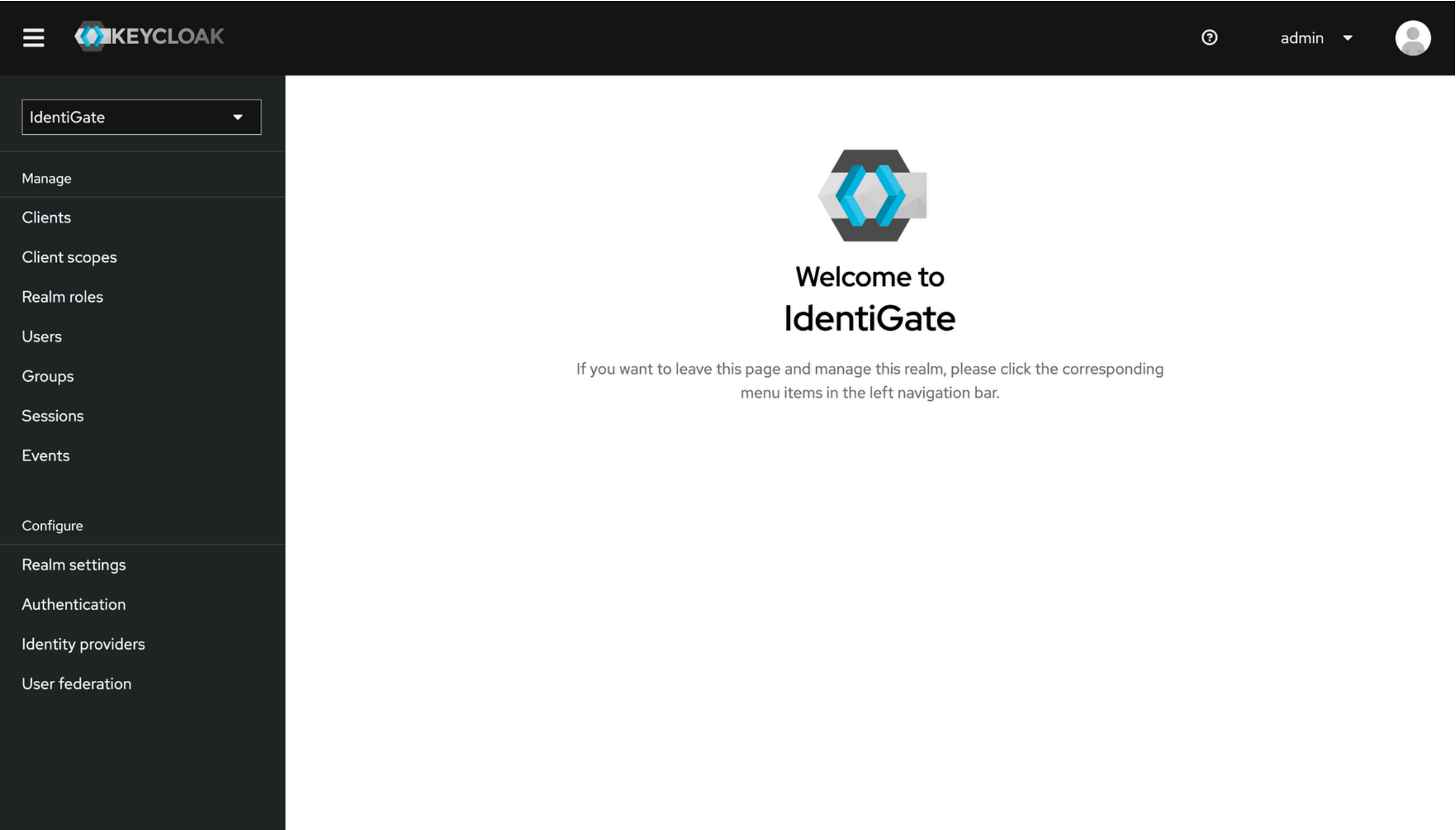
Une fois l'environnement Docker opérationnel, la configuration de Keycloak a été réalisée :

- Création du realm IdentiGate : un espace d'authentification isolé dédié au projet
- Ajout de clients :
 - admin-cli-api pour l'administration via API
 - Ardo pour simuler un client applicatif
- Définition des rôles : ex. stagiaire, expert-cyber, partenaire-x
- Création des utilisateurs et assignation des rôles correspondants



Initialisation de l'environnement Keycloak(2/3)

Déploiement



Interface admin Keycloak

Initialisation de l’environnement Keycloak (3/3)

Déploiement

KEYCLOAK

admin

IdentiGate

Manage

Clients

Client scopes

Realm roles

Users

Groups

Sessions

Events

Configure

Realm settings

Authentication

Identity providers

User federation

Clients

Clients are applications and services that can request authentication of a user. [Learn more](#)

Clients listInitial access tokenClient registration

Search for client

Create client

Import client

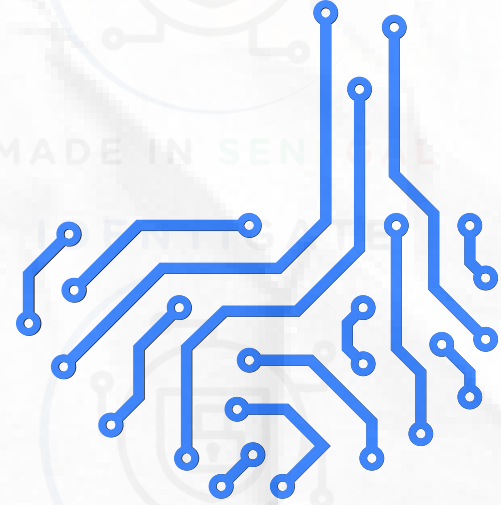
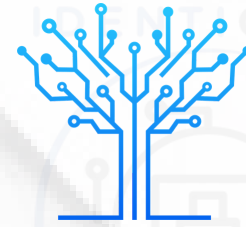
Refresh

1 - 9

Client ID	Name	Type	Description	Home URL
account	\${client_account}	OpenID Connect	–	http://localhost:8280/realms/IdentiGate/account/
account-console	\${client_account-console}	OpenID Connect	–	http://localhost:8280/realms/IdentiGate/account/
admin-cli	\${client_admin-cli}	OpenID Connect	–	–
admin-cli-api	–	OpenID Connect	–	–
Ardo	Abdoul Ardo KAH	OpenID Connect	–	–
broker	\${client_broker}	OpenID Connect	–	–
identigate-frontend	–	OpenID Connect	–	http://localhost:3000
realm-management	\${client_realm-managemen...}	OpenID Connect	–	–
security-admin-console	\${client_security-admin-co	OpenID		http://localhost:8280/admin/IdentiGate/console/



Listes des clients créés
dont “Ardo et Admin-cli-
api



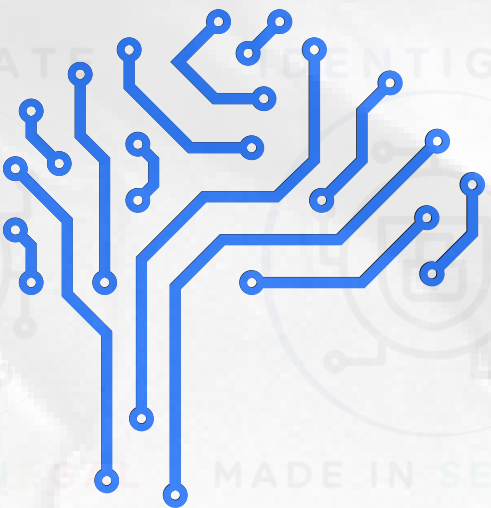
IV

Provisioning & Sécurité

Provisionnement automatique des
utilisateurs - Mise en place de
l'authentification forte(2FA)

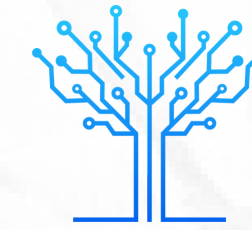


Gestion des Identités et
Accès



Provisionnement automatique des utilisateurs (1/3)

Provisioning & Sécurité



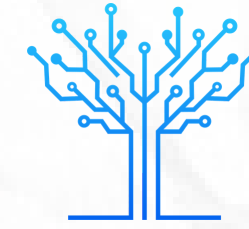
Afin d'optimiser la gestion des utilisateurs et de gagner du temps lors de la configuration initiale du realm, un script bash intitulé `provision-users.sh` a été développé. Ce script permet la création automatisée d'utilisateurs via l'API REST de Keycloak. Il s'intègre facilement dans le processus de déploiement, en ajoutant les utilisateurs nécessaires avec les rôles appropriés, sans passer par l'interface d'administration.

Parmi les comptes créés automatiquement, on retrouve notamment :

- stagiaire1 (profil utilisateur basique)
- expert-cyber (profil expert avec droits étendus)
- partenaire-x (profil externe ou partenaire)

Provisionnement automatique des utilisateurs (2/3)

Provisioning & Sécurité



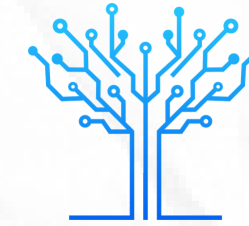
Afin d'optimiser la gestion des utilisateurs et de gagner du temps lors de la configuration initiale du realm, un script bash intitulé `provision-users.sh` a été développé. Ce script permet la création automatisée d'utilisateurs via l'API REST de Keycloak. Il s'intègre facilement dans le processus de déploiement, en ajoutant les utilisateurs nécessaires avec les rôles appropriés, sans passer par l'interface d'administration.

Parmi les comptes créés automatiquement, on retrouve notamment :

- stagiaire1 (profil utilisateur basique)
- expert-cyber (profil expert avec droits étendus)
- partenaire-x (profil externe ou partenaire)

Provisionnement automatique des utilisateurs (3/3)

Provisioning & Sécurité



Ce provisionnement automatique permet également d'assigner les rôles dès la création du compte, assurant une gestion cohérente des accès selon les profils définis.

```
1  #!/bin/bash
2
3  # --- CONFIGURATION ---
4  KEYCLOAK_URL="http://localhost:8280"
5  REALM="IdentiGate"
6  CLIENT_ID="admin-cli-api"
7  CLIENT_SECRET="nSYegfqJiuBJZRcMK57taC9hgZmcZlxx"
8
9  # Liste des utilisateurs à créer (username;email;password;role)
10 USERS=(
11     "stagiaire1;stagiaire1@example.com;stagiaire123;user"
12     "expert-cyber;cyber@example.com;expert2024;admin"
13     "partenaire-x;partner@example.com;partenaire123;partenaire"
14 )
15
16 # --- OBTENTION DU TOKEN ---
17 echo "🔑 Obtention du token..."
18
19 RESPONSE=$(curl -s -X POST "$KEYCLOAK_URL/realms/$REALM/protocol/openid-connect/token" \
20     -H "Content-Type: application/x-www-form-urlencoded" \
21     -d "grant_type=client_credentials" \
22     -d "client_id=$CLIENT_ID" \
23     -d "client_secret=$CLIENT_SECRET")
24
25 echo "📄 Réponse token brute : $RESPONSE"
26
27 ACCESS_TOKEN=$(echo "$RESPONSE" | grep -o '"access_token": "[^"]*" | cut -d ':' -f2 | tr -d '"')
28
29 if [[ -z "$ACCESS_TOKEN" || "$ACCESS_TOKEN" == "null" ]]; then
30     echo "❌ Échec de l'obtention du token."
31     exit 1
32 fi
33
34 echo "✅ Token récupéré avec succès !"
35
36 # --- PROVISIONNEMENT DES UTILISATEURS ---
37 for entry in "${USERS[@]"; do
38     IFS=";" read -r username email password role <<< "$entry"
39
40     echo ""
41     echo "👤 Création de l'utilisateur : $username"
42
43     JSON_PAYLOAD=$(cat <<EOF
44 {
45     "username": "$username",
46     "email": "$email",
47     "enabled": true,
48     "credentials": [{
49         "type": "password",
50         "value": "$password",
51         "temporary": false
52     }]
53 }
54 EOF
55 )
56
57 echo "📤 Requête envoyée : "
58 echo "$JSON_PAYLOAD"
59
60 RESPONSE=$(curl -s -w "%{http_code}" -X POST "$KEYCLOAK_URL/admin/realms/$REALM/users" \
61     -H "Authorization: Bearer $ACCESS_TOKEN" \
62     -H "Content-Type: application/json" \
63     -d "$JSON_PAYLOAD")
64
65 # Extraire le code HTTP
66 HTTP_CODE=$(echo "$RESPONSE" | grep HTTP_CODE | cut -d ':' -f2)
67 BODY=$(echo "$RESPONSE" | sed '/HTTP_CODE/d')
68
69 if [[ "$HTTP_CODE" == "201" ]]; then
70     echo "✅ Utilisateur $username créé avec succès !"
71 else
72     echo "❌ Erreur pour $username (HTTP $HTTP_CODE)"
73     echo "📄 Détail de l'erreur : $BODY"
74 fi
75 done
```

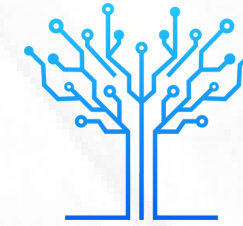
```
36 # --- PROVISIONNEMENT DES UTILISATEURS ---
37 for entry in "${USERS[@]"; do
38     IFS=";" read -r username email password role <<< "$entry"
39
40     echo ""
41     echo "👤 Création de l'utilisateur : $username"
42
43     JSON_PAYLOAD=$(cat <<EOF
44 {
45     "username": "$username",
46     "email": "$email",
47     "enabled": true,
48     "credentials": [{
49         "type": "password",
50         "value": "$password",
51         "temporary": false
52     }]
53 }
54 EOF
55 )
56
57 echo "📤 Requête envoyée : "
58 echo "$JSON_PAYLOAD"
59
60 RESPONSE=$(curl -s -w "%{http_code}" -X POST "$KEYCLOAK_URL/admin/realms/$REALM/users" \
61     -H "Authorization: Bearer $ACCESS_TOKEN" \
62     -H "Content-Type: application/json" \
63     -d "$JSON_PAYLOAD")
64
65 # Extraire le code HTTP
66 HTTP_CODE=$(echo "$RESPONSE" | grep HTTP_CODE | cut -d ':' -f2)
67 BODY=$(echo "$RESPONSE" | sed '/HTTP_CODE/d')
68
69 if [[ "$HTTP_CODE" == "201" ]]; then
70     echo "✅ Utilisateur $username créé avec succès !"
71 else
72     echo "❌ Erreur pour $username (HTTP $HTTP_CODE)"
73     echo "📄 Détail de l'erreur : $BODY"
74 fi
75 done
```



Script bash du
provisionnement des
utilisateurs

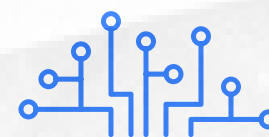
Mise en place de l'authentification forte(2FA) (1/2)

Provisioning & Sécurité



Dans un souci de renforcement de la sécurité, une authentification à deux facteurs (2FA) a été mise en place. Cette mesure de sécurité oblige l'utilisateur à fournir une preuve d'identité supplémentaire lors de la connexion, en plus du mot de passe.

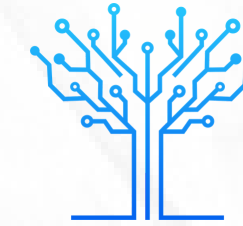
Pour cela, Google Authenticator a été utilisé. Lors de sa première connexion, l'utilisateur scanne un QR code affiché par Keycloak, ce qui associe son compte à l'application mobile. À chaque connexion suivante, il devra fournir un code temporaire à usage unique (OTP) généré par l'application.



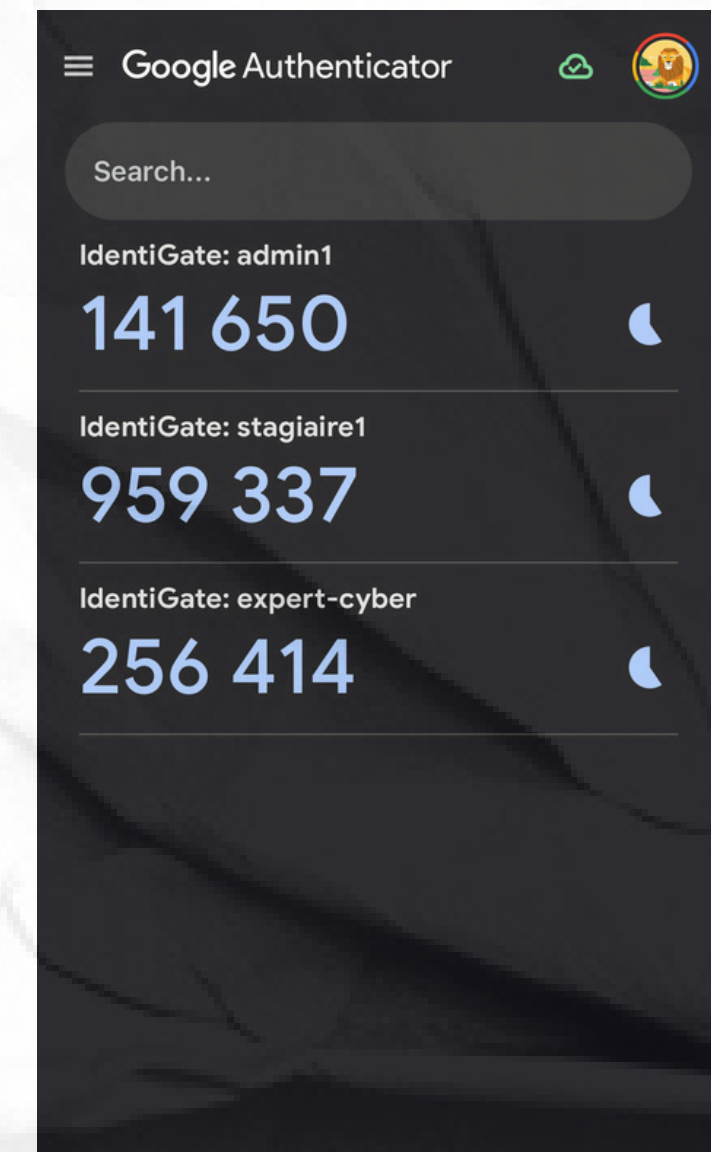
Script bash du
provisionnement des
utilisateurs

Mise en place de l'authentification forte(2FA) (2/2)

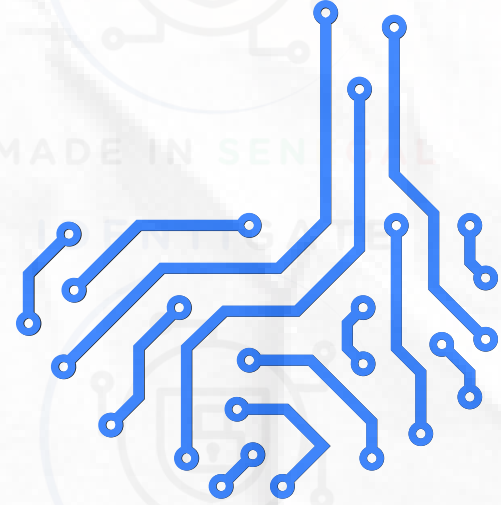
Provisioning & Sécurité



Cette méthode permet de réduire significativement les risques liés aux mots de passe compromis et de mieux protéger les accès sensibles au sein de l'infrastructure.



One time code
Google Authenticator



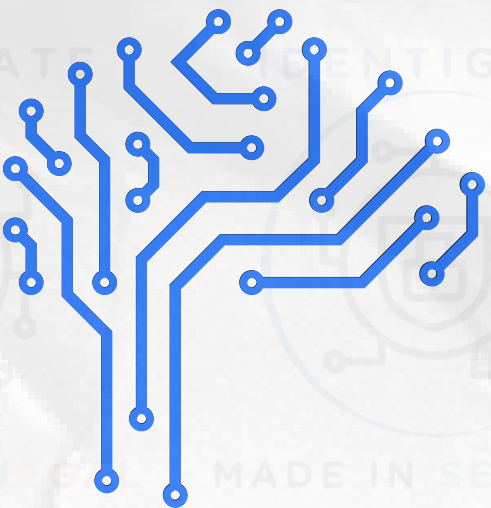
V

Gestion des rôles

Création et gestion de rôles
personnalisés - Assignment
automatique via script - Affichage
dynamique côté frontend

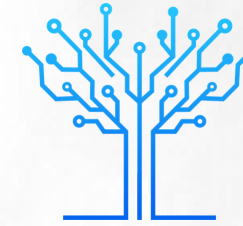


Gestion des Identités et
Accès



Création et gestion de rôles personnalisés

Gestion des rôles



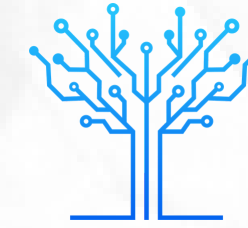
Dans le cadre du projet, des rôles personnalisés ont été créés au sein du realm IdentiGate pour refléter différents types d'utilisateurs. Ces rôles permettent de contrôler précisément les droits d'accès à certaines fonctionnalités ou ressources.

Les rôles définis sont les suivants :

- admin : dispose de tous les privilèges d'administration
- partenaire : représente un utilisateur externe ou collaborateur
- user : profil utilisateur standard, avec des droits limités

Assignation automatique via script(1/2)

Gestion des rôles

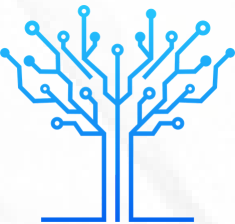


L'assignation de ces rôles aux utilisateurs se fait automatiquement grâce au script Bash `provision-users.sh`. Lors de la création d'un utilisateur, le rôle approprié lui est directement attribué via l'API REST de Keycloak, sans intervention manuelle dans l'interface graphique.

Ce mécanisme assure une cohérence dans la gestion des droits dès l'intégration des comptes dans le système.

Assignment automatique via script (2/2)

Gestion des rôles



```
1 #!/bin/bash
2
3 # --- CONFIG ---
4 KEYCLOAK_URL="http://localhost:8280"
5 REALM="IdentiGate"
6 ACCESS_TOKEN="eyJhbGciOiJSUzI1NiIsInR5cCIgOiAiSldUIiwia2lkIiA6ICJFd0NgbXNSVG1DVHRscUx1NURpYVViMkNjM2ZQcEdoNmtERlNkTVlMNUI4In0.ey"
7
8 # Définition des utilisateurs et rôles via 2 tableaux
9 USERS=("stagiaire1" "expert-cyber" "partenaire-x")
10 ROLES=("user" "admin" "partenaire")
11
12 # Nombre d'éléments
13 N=${#USERS[@]}
14
15 for (( i=0; i<N; i++ )); do
16     username=${USERS[i]}
17     role=${ROLES[i]}
18
19     echo "🔍 Recherche de l'ID utilisateur : $username"
20     USER_ID=$(curl -s -H "Authorization: Bearer $ACCESS_TOKEN" \
21         "$KEYCLOAK_URL/admin/realms/$REALM/users?username=$username" | \
22         grep -o '"id":"[^"]*" | cut -d':' -f2 | tr -d '"' )
23
24     if [ -z "$USER_ID" ]; then
25         echo "❌ Utilisateur $username non trouvé !"
26         continue
27     fi
28
29     echo "🔍 Recherche des infos du rôle : $role"
30     ROLE_JSON=$(curl -s -H "Authorization: Bearer $ACCESS_TOKEN" \
31         "$KEYCLOAK_URL/admin/realms/$REALM/roles/$role")
32
33     echo "📝 Assignment du rôle $role à $username ($USER_ID)..."
34     curl -s -X POST "$KEYCLOAK_URL/admin/realms/$REALM/users/$USER_ID/role-mappings/realms/" \
35         -H "Authorization: Bearer $ACCESS_TOKEN" \
36         -H "Content-Type: application/json" \
37         -d "$ROLE_JSON"
38
39     echo "✅ Rôle $role assigné à $username"
40 done
41
```

Realm roles

Realm roles are the roles that you define for use in the current realm. [Learn more](#)

1-9

< >

Role name	Composite	Description
admin	False	—
default-roles-identigate ⓘ	True	\${role_default-roles}
expert	False	—
offline_access	False	\${role_offline-access}
partenaire	False	—
Partenaire	False	—
stagiaire	False	—
uma_authorization	False	\${role_uma_authorization}
user	False	—



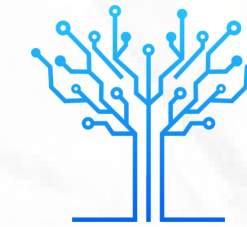
Script bash du provisioning des rôles



Présence des rôles dans l'interface

Affichage dynamique côté frontend(1/3)

Gestion des rôles



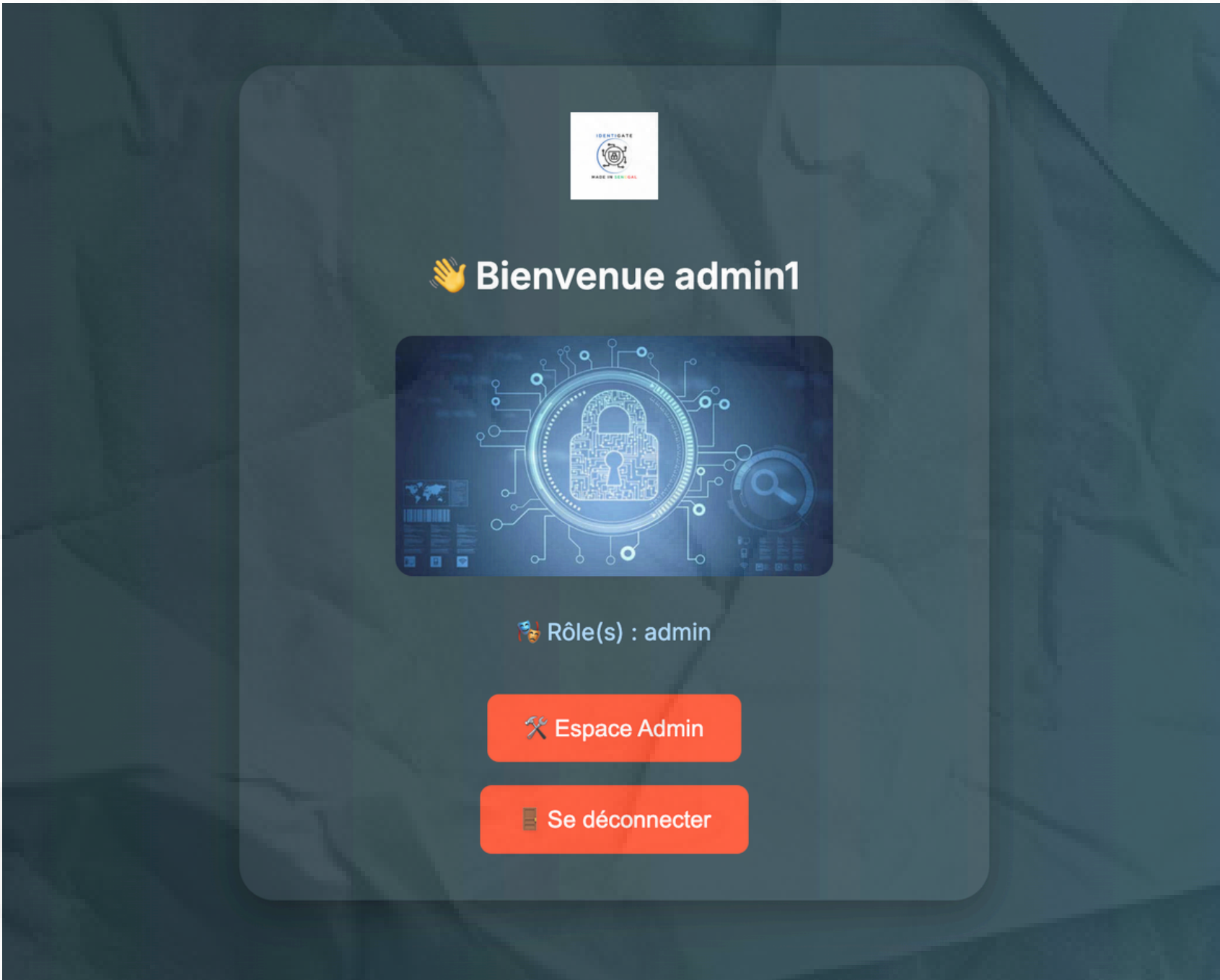
Côté interface utilisateur, un script JavaScript côté client permet de récupérer le rôle de l'utilisateur connecté via le token Keycloak. En fonction de ce rôle, le contenu de la page HTML s'adapte dynamiquement :

- Certaines sections sont affichées ou masquées,
- Des messages personnalisés sont affichés selon le profil,
- Des actions spécifiques sont rendues disponibles ou désactivées.

Ce fonctionnement garantit une expérience utilisateur adaptée tout en maintenant une sécurité par contrôle d'accès côté client.

Affichage dynamique côté frontend(2/3)

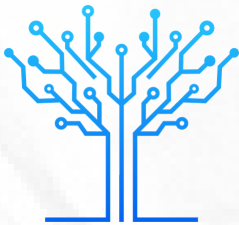
Gestion des rôles



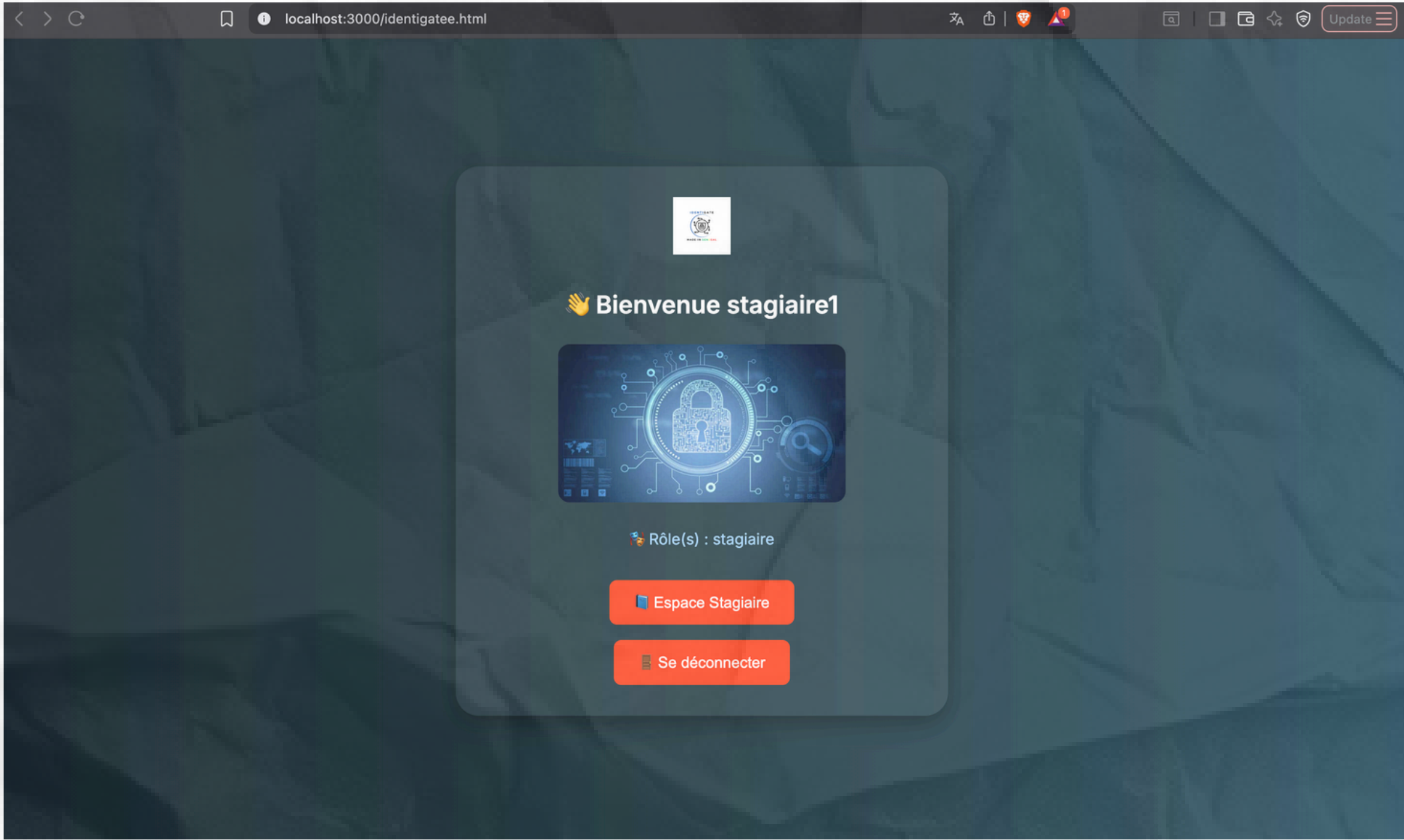
Connecté en tant que
administrateur

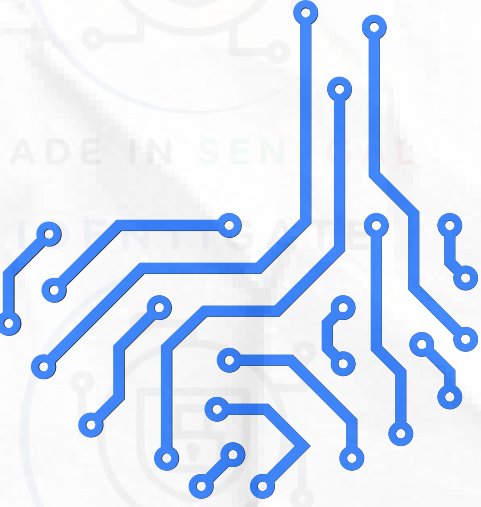
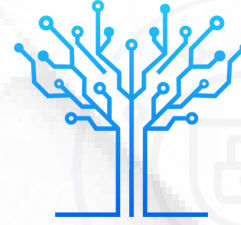
Affichage dynamique côté frontend(3/3)

Gestion des rôles



Connecté en tant que stagiaire





VI

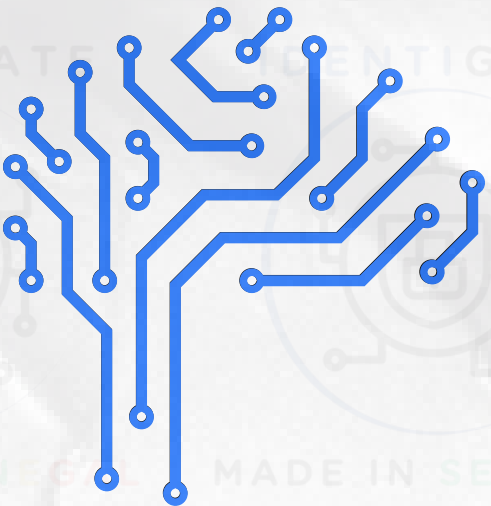
Single Sign-On

SSO

Intégration du SSO - Connexion
centralisée
- Déconnexion synchronisée

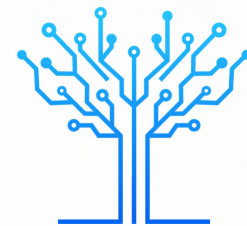


Gestion des Identités et
Accès



Intégration du SSO

SSO



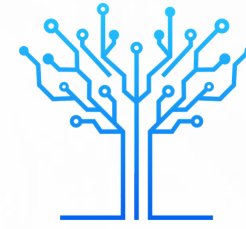
Dans le cadre du projet, deux applications web (IdentiGate.html et expert.html) ont été sécurisées à l'aide de Keycloak.js, la bibliothèque officielle de Keycloak pour les clients JavaScript.



One time code
Google Authenticator

Connexion centralisée

SSO

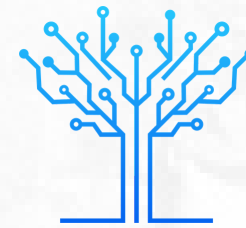


Une fois l'utilisateur connecté à l'une des applications, il peut accéder à l'autre sans avoir à se reconnecter.

Cela offre une expérience fluide et cohérente, conforme aux principes du SSO.

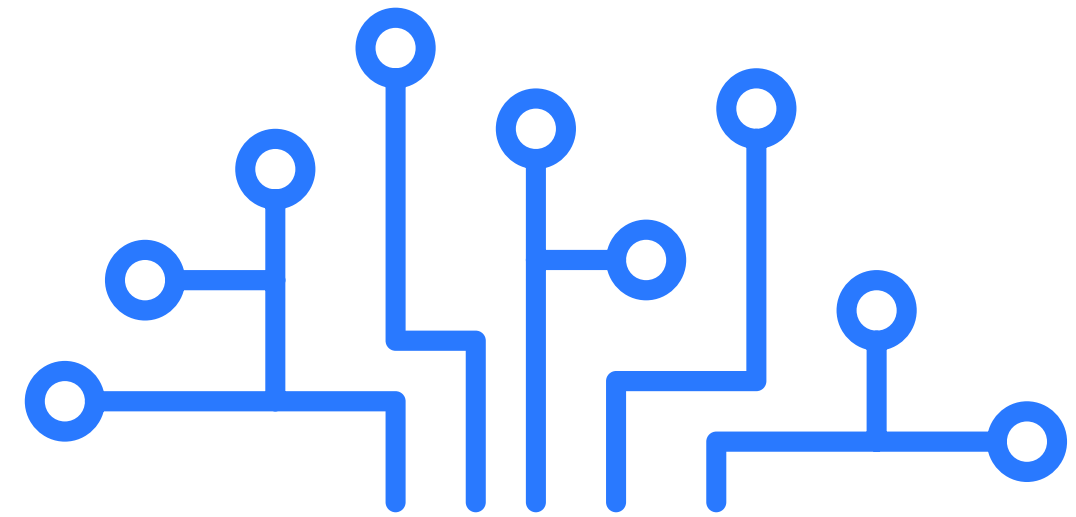
Déconnexion synchronisée

SSO

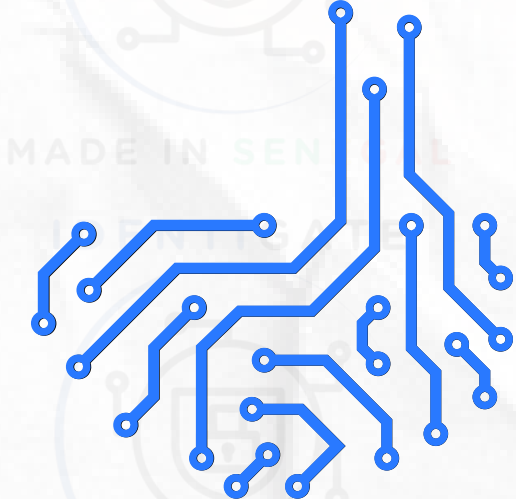


Lorsque l'utilisateur se déconnecte depuis l'une des interfaces, la session est automatiquement terminée sur toutes les applications protégées.

Ce mécanisme assure une meilleure sécurité et simplifie la gestion des sessions.



Démonstration



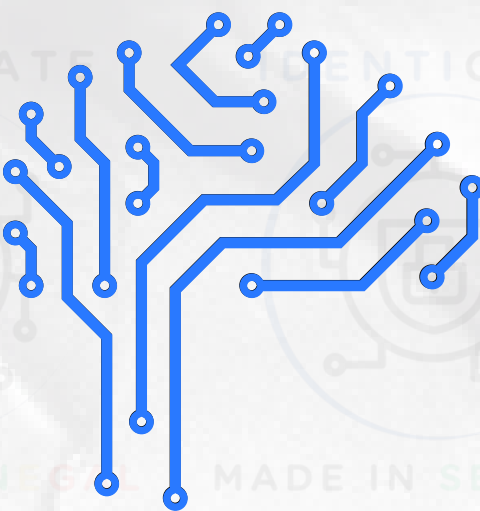
VII

Export & tests

Docker - Initialisation de
l'environnement Keycloak

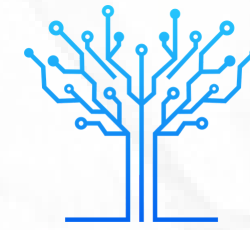


Gestion des Identités et
Accès



Export de la configuration et phase de tests

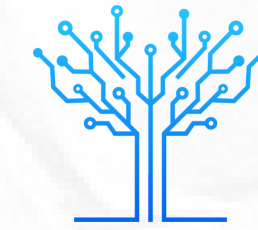
Export, tests & sécurité



Une fois l'environnement Keycloak entièrement configuré (realm, clients, utilisateurs, rôles), la configuration a été exportée au format JSON. Cela permet de sauvegarder, versionner et réutiliser facilement la configuration du realm, notamment en cas de déploiement sur un autre environnement ou pour la reprise du projet.

Tests avec différents profils utilisateurs

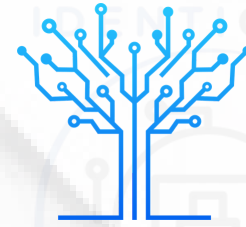
Export, tests & sécurité



Des scénarios de test ont été menés avec les différents types d'utilisateurs créés :

- Le rôle user est volontairement restreint. Il ne peut pas accéder aux pages ou fonctionnalités réservées à des rôles supérieurs (comme l'administration).
- Le rôle admin, quant à lui, bénéficie de l'accès complet à l'ensemble des ressources.

Ces tests ont permis de valider la cohérence du modèle de rôles et la bonne application des restrictions selon les droits.



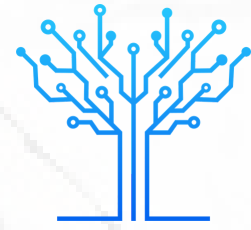
VIII

Conclusion

Conclusion - Perspective



Gestion des Identités et
Accès



Conclusion

Le projet IdentiGate a permis de démontrer, à travers un cas pratique, l'intérêt d'une solution IAM bien structurée pour assurer la protection des ressources informatiques.

En combinant Keycloak, des scripts automatisés, et une interface frontale adaptée, le système mis en place offre un niveau de sécurité robuste tout en garantissant une expérience utilisateur fluide.

Cette démarche contribue à répondre aux enjeux actuels de cybersécurité, tout en posant les bases d'une infrastructure IAM scalable et personnalisable.

Ce projet ouvre également des perspectives d'évolution, notamment en intégrant des outils comme LDAP ou des connecteurs externes pour aller plus loin dans la fédération des identités.